

04

강

R 컴퓨팅

R데이터의 구조 (1)

통계·데이터과학과 장영재 교수

학습목차

1 데이터의 기본 형태

2 벡터

3 행렬

4 실습하기

학습목표

- 1 벡터의 특성을 이해한다.
- 2 행렬의 특성을 이해한다.
- 3 벡터 및 행렬의 유사성과 차이점을 이해한다.
- 4 벡터 및 행렬을 생성할 수 있다.
- 5 벡터 및 행렬과 관련한 연산을 실행할 수 있다.





데이터의 기본 형태

1. 데이터의 기본 형태

▶ R의 특징 중 하나는 다양한 형태의 데이터 입수와 가공이 용이하며 통계분석에 최적화된 형태의 데이터 구조를 형성한다는 점임

- R에서 다루는 데이터의 기본적인 형태는 크게

벡터, 행렬, 배열과 **리스트, 데이터프레임**으로 나눌 수 있음

02

벡터

2. 벡터

▶ 벡터의 생성

- 벡터는 1개 이상의 원소로 구성된 자료구조로, R의 자료 객체 중에서 가장 기본이 되는 자료 객체임
- 벡터 하나의 원소는 한 가지 형태(mode)만이 가능하다는 점에 유의
- 벡터는 `c()`, `scan()`, `seq()`, `rep()` 함수를 이용하여 생성 가능

예제 3-1 `c()` 함수를 이용하여 3에서 7까지의 자연수로 이루어진 벡터를 생성

```
> c(3,4,5,6,7)      # c() 함수 내에 입력된 값을 조합하여 벡터 생성
```

```
[1] 3 4 5 6 7
```

```
> c(3:7)             # c() 함수 내 수열 값을 입력하여 벡터 생성
```

```
[1] 3 4 5 6 7
```

```
> c(7,3,5,4,6)      # 순서 임의로 지정 가능
```

```
[1] 7 3 5 4 6
```

예제 3-2 `c()` 함수를 이용하여 "1", "KOREA", "NATIONAL", "OPEN", "UNIVERSITY"로 이루어진 문자형 벡터를 생성

```
> c("1","KOREA","NATIONAL","OPEN","UNIVERSITY")
```

```
[1] "1"      "KOREA"   "NATIONAL" "OPEN"    "UNIVERSITY"
```

문자형 벡터 생성 시에는 큰따옴표(" ")를 반드시 추가해야 함(필수적으로 추가됨)

2. 벡터

예제 3-3 [예제 3-2]에서 생성한 벡터를 v1이란 이름으로 저장하고 length() 함수를 사용하여 이 벡터의 크기를 출력

```
> v1 <- c("1","KOREA","NATIONAL","OPEN","UNIVERSITY")  
> length(v1)  
[1] 5
```

예제 3-4 c() 함수를 이용하여 논리형 벡터를 생성

```
> c(F,F,T)  
[1] FALSE FALSE TRUE  
> c(TRUE,FALSE,TRUE)  
[1] TRUE FALSE TRUE
```


2. 벡터

예제 3-5 scan() 함수를 이용하여 3에서 7까지 자연수로 이루어진 벡터를 생성

```
> scan() # 벡터 생성을 위한 scan() 함수 입력
```

```
1: 3 4 5 # 벡터의 첫 번째부터 세 번째 원소들을 입력
```

```
4: 6 7 # 벡터의 네 번째, 다섯 번째 원소들을 입력
```

```
6: # 공백
```

```
Read 5 items
```

```
[1] 3 4 5 6 7
```

```
> scan(sep=",") # 벡터 생성을 위한 scan() 함수 입력
```

```
1: 3 # 벡터의 첫 번째 원소를 입력
```

```
2: 4,5 # 벡터의 두 번째와 세 번째 원소를 입력
```

```
4: 6,7 # 벡터의 네 번째부터 다섯 번째 원소를 입력
```

```
6: # 공백
```

```
Read 5 items
```

```
[1] 3 4 5 6 7
```

2. 벡터

예제 3-6 seq() 함수를 이용하여 3에서 7까지의 자연수로 이루어진 벡터를 생성

```
> seq(from=3, to=7, by=1)      # from=시작값, to=종료값, by=증가분
```

```
[1] 3 4 5 6 7
```

```
> seq(to=7, from=3, by=1)      # to=종료값, from=시작값, by=증가분
```

```
[1] 3 4 5 6 7
```

```
> seq(by=1, from=3, to=7)      # by=증가분, from=시작값, to=종료값
```

```
[1] 3 4 5 6 7
```

```
> seq(3, 7, 1)  # 생략 시 from=시작값, to=종료값, by=증가분 순서로 인식
```

```
[1] 3 4 5 6 7
```

- 벡터를 구성하는 수열의 시작과 끝을 나타내는 from, to,
그리고 간격을 의미하는 by 값이 지정되면 그 순서에 상관없이 동일한 벡터를 생성

2. 벡터

예제 3-7 seq() 함수의 조건을 지정하여 벡터를 생성

```
> seq(from=3, to=7, length=3)    # from=시작값, to=종료값, 조건지정
```

```
[1] 3 5 7
```

```
> seq(from=1, by=0.05, along=1:5)    # from=시작값, by=증가분, 조건지정
```

```
[1] 1.00 1.05 1.10 1.15 1.20
```

```
> seq(from=1, to=5, along=1:6)    # from=시작값, to=종료값, 조건지정
```

```
[1] 1.0 1.8 2.6 3.4 4.2 5.0
```

- length로 조건을 지정할 경우에는 벡터의 크기, 즉 원소의 개수를 지정하여 시작값과 종료값 사이를 등간격으로 나누어 벡터를 생성
- along으로 수열을 지정하게 되면 해당 수열에 따라 동일한 크기의 벡터를 생성

2. 벡터

예제 3-8 c() 함수를 이용하여 기본 벡터를 생성한 뒤 rep() 함수를 이용하여
생성된 벡터의 일정 원소가 반복되는 벡터

```
> rep(c(1,2), times=2) # 1과 2로 구성된 벡터를 2회 반복
```

```
[1] 1 2 1 2
```

```
> rep(1:2, times=2) # 1:2 1에서 2까지로 구성된 벡터를 2회 반복
```

```
[1] 1 2 1 2
```

```
> rep(c(2,4), times=c(2,1)) # 앞의 원소는 2회, 뒤의 원소는 1회 반복
```

```
[1] 2 2 4
```

```
> rep(c(2,4), each=2) # 앞, 뒤 모든 원소를 2회 반복
```

```
[1] 2 2 4 4
```

```
> rep(c(2,4,8), length=5) # 2, 4, 8로 구성된 벡터를 반복하되 생성되는 벡
```

```
# 터의 크기를 5로 한정
```

```
[1] 2 4 8 2 4
```

2. 벡터

▶ 벡터 자료의 편집

- 벡터와 관련된 기본함수를 통해 벡터를 생성하였다면, 그에 따라서는 이를 적당히 편집하고 가공할 필요가 있음

<자료의 삽입, 삭제 등과 관련된 방법들>

- 1) []를 이용하여 벡터의 일부 원소를 추출
- 2) replace() 함수를 이용하여 일부 자료를 대체
- 3) append() 함수를 이용하여 조건에 따른 위치에 자료를 삽입
- 4) sort() 함수를 이용하여 자료를 정렬
- 5) rank() 함수를 이용하여 자료의 순위를 출력
- 6) order() 함수를 이용하여 오름차순에 의한 자료의 위치값 출력

2. 벡터

벡터명[원소번호 또는 조건문, ...]

`replace(벡터, 원소번호, 교체자료)`

`append(벡터, 삽입자료, after=원소번호)`

`sort(벡터, decreasing=FALSE, ...)`

`rank(벡터, na.last=TRUE, ties.method=c("average", "first", "random", "max", "min"))`

`order(벡터, na.last=TRUE, decreasing=FALSE)`

- “na.last=TRUE”라고 기본값으로 지정된 옵션의 의미는 만약 결측치가 존재한다면 이러한 결측치들을 자료의 맨 끝에 위치하는 것으로 간주한다는 의미
- “decreasing=FALSE”라는 기본 옵션은 내림차순이 아니라는 즉, 오름차순을 기본값으로 한다는 의미

2. 벡터

예제 3-9 ① 11에서 20까지의 값을 갖는 벡터 v1을 생성하고 세 번째와 다섯 번째 값을 추출,

② v1의 원소 중 15보다 큰 값만을 출력하고 v1 중 두 번째와 네 번째 값을 삭제하고 출력

```
> v1 <- c(11:20) # 11에서 20까지의 값을 갖는 벡터 v1 생성
```

```
> v1
```

```
[1] 11 12 13 14 15 16 17 18 19 20
```

```
> v1[c(3,5)]      # v1의 세 번째와 다섯 번째 값을 출력
```

```
[1] 13 15
```

```
> v1[v1 > 15]     # v1의 원소 중 15보다 큰 값을 출력
```

```
[1] 16 17 18 19 20
```

```
> v1[c(-2,-4)]     # v1의 원소 중 두 번째와 네 번째 값을 삭제하고 출력
```

```
[1] 11 13 15 16 17 18 19 20
```

2. 벡터

예제 3-10 ① 3에서 7까지의 값을 갖는 벡터 v2를 생성하고 replace() 함수를 이용하여 v2의 두 번째 원소를 10으로 바꾸어 출력, ② append() 함수를 이용하여 v2의 다섯 번째 자료 다음에 8을 삽입하여 출력

```
> v2 <- c(3:7)
```

```
> v2
```

```
[1] 3 4 5 6 7
```

```
> replace(v2,2,10) # v2의 두 번째 원소의 값을 10으로 변경
```

```
[1] 3 10 5 6 7
```

```
> append(v2,8,after=5) # v2의 다섯 번째 값 다음에 8을 추가
```

```
[1] 3 4 5 6 7 8
```

```
> v3 <- append(v2,8,after=5)
```

```
> v3
```

```
[1] 3 4 5 6 7 8
```


2. 벡터

예제 3-11 다음의 '규칙'에 따라 생성된 벡터 x를 ① sort() 함수를 이용하여 오름차순으로 정렬하고 rank() 함수를 통해 자료의 순위를 출력, ② order() 함수를 이용하여 오름차순에 의한 자료의 위치값을 출력
<규칙> 3을 세 번 반복한 뒤 3에서 7까지 2만큼의 증가분으로 수열을 생성하고 이어서 rev() 함수를 이용해 3에서 7까지 이르는 크기 3인 벡터의 역순을 구해 연결한 뒤 4를 세 번 반복한 벡터를 덧붙임

```
> x <- c(rep(3,3),seq(3,7,by=2),rev(seq(3,7,length=3)),rep(4,3))
```

```
> x
```

```
[1] 3 3 3 3 5 7 7 5 3 4 4 4
```

```
> sort(x)
```

벡터 x를 오름차순으로 정렬

```
[1] 3 3 3 3 3 4 4 4 5 5 7 7
```

```
> rank(x)
```

자료의 순위를 출력

```
[1] 3.0 3.0 3.0 3.0 9.5 11.5 11.5 9.5 3.0 7.0 7.0 7.0
```

```
> order(x)
```

오름차순에 의한 자료의 위치값 출력

```
[1] 1 2 3 4 9 10 11 12 5 8 6 7
```

2. 벡터

<sort() 함수와 order() 함수의 기능>

```
> x
[1] 3 3 3 3 5 7 7 5 3 4 4 4

> rank(x)
[1] 3.0 3.0 3.0 3.0 9.5 11.5 11.5 9.5 3.0 7.0 7.0 7.0

> sort(x)
[1] 3 3 3 3 3 4 4 4 5 5 7 7

> order(x) # 오름차순에 의한 자료의 위치값 출력
[1] 1 2 3 4 9 10 11 12 5 8 6 7
```

2. 벡터

<벡터와 관련된 여러 가지 함수>

함수	설명	사용 예
scan	자료를 화면에서 직접 입력	scan(), scan("data")
c	자료를 조합하여 벡터 생성	c(1, 2, 3, 4), c("yes", "no")
rep	자료를 반복하여 벡터 생성	rep(NA, 5), rep(c(1, 2), 3)
:	수치적 수열을 나타냄	1:5, 1:-1
seq	수치적 수열을 나타냄	seq(-pi, pi, 5)
vector	벡터로 초기화	vector("complex", 5)
logical	logical 벡터로 초기화	logical(3)
numeric	numeric 벡터로 초기화	numeric(4)
complex	complex 벡터로 초기화	complex(5)
character	character 벡터로 초기화	character(6)

03

행렬

3. 행렬

- ▶ 행렬은 동일한 형태로 구성된 2차원의 데이터 구조
 - 행의 차원과 열의 차원을 갖고 있으며, 벡터와 마찬가지로 하나의 행렬은 수치형, 문자형, 논리형 중 한 가지 형태의 원소만 갖는다는 점에 유의

<행렬의 주요 속성>

속성	설명
length	자료의 개수
mode	자료의 형태
dim	행과 열의 개수
dimnames	행과 열의 이름

3. 행렬

예제 3-12 matrix() 함수를 이용하여 1에서 9까지의 원소를 갖는 3행 3열의 행렬을 생성하고 출력

```
> matr <- matrix(1:9, nrow=3)      # 3행 3열 행렬 생성
```

```
> matr
```

```
[,1] [,2] [,3]
```

```
[1,] 1 4 7
```

```
[2,] 2 5 8
```

```
[3,] 3 6 9
```

```
> length(matr)                    # 원소의 개수
```

```
[1] 9
```

```
> mode(matr)                     # 원소의 형태
```

```
[1] "numeric"
```

```
> dim(matr)                      # 행과 열의 개수
```

```
[1] 3 3
```

3. 행렬

▶ 행렬의 생성

- 행렬은 여러 변수들이 이차원적으로 모여 있는 객체
- 행렬을 생성하는 간단한 기본함수
 - 1) matrix() 함수를 이용하여 직접 생성
 - 2) cbind() 함수를 이용하여 벡터를 열(column)을 기준으로 병합
 - 3) rbind() 함수를 이용하여 벡터를 행(row)을 기준으로 병합
 - 4) dim() 함수를 이용하여 차원(행과 열의 개수)을 직접 지정

```
matrix(data, nrow=, ncol=, byrow=FALSE, dimnames=NULL)
```

```
cbind(벡터1, 벡터2, ...) 또는 rbind(벡터1, 벡터2, ...)
```

```
dim(x) <- c(행의 개수, 열의 개수)
```

3. 행렬

예제 3-13 [예제 3-12]에서 matrix() 함수를 이용하여 생성된 행렬과 동일한 형태의 행렬을 rbind(), cbind(), dim() 함수를 이용하여 생성

```
> r1 <- c(1,2,3)                # r1, r2, r3 행 벡터 생성
> r2 <- c(4,5,6)
> r3 <- c(7,8,9)
> rbind(r1, r2, r3)             # rbind: 행을 기준으로 결합
[,1] [,2] [,3]
r1 1 2 3
r2 4 5 6
r3 7 8 9
> c1 <- c(1:3)                  # c1, c2, c3 열 벡터 생성
> c2 <- c(4:6)
> c3 <- c(7:9)
```


3. 행렬

> cbind(c1, c2, c3) # cbind: 열을 기준으로 결합

c1 c2 c3

[1,] 1 4 7

[2,] 2 5 8

[3,] 3 6 9

> m1 <- 1:9 # :만으로도 c(1:9)와 동일한 기능을 수행

> dim(m1) <- c(3,3) # dim: 차원을 지정

> m1

[,1] [,2] [,3]

[1,] 1 4 7

[2,] 2 5 8

[3,] 3 6 9

3. 행렬

▶ 행렬의 연산

- 벡터와 마찬가지로 행렬의 경우에도 자료의 삽입, 삭제, 행 또는 열의 연산 등과 관련된 다양한 방법들이 존재

- 주요 함수의 예

1) []를 이용하여 행렬의 일부 원소를 추출

2) apply() 함수를 이용한 행 또는 열의 연산

3) sweep() 함수를 이용한 행 또는 열의 연산

행렬명[원소번호 또는 조건문, ...]

apply(행렬, 조건(1은 행, 2는 열, c(1,2)는 행과 열), FUN, ...)

sweep(행렬, 조건(1은 행, 2는 열, c(1,2)는 행과 열), STATS, FUN="-", ...)

3. 행렬

예제 3-14 1에서 9까지의 원소를 갖는 행 기준의 3행 3열 행렬을 만들고 특정 행과 열,

조건에 따른 원소 추출들을 실행하고 `apply()`와 `sweep()` 함수를 통한 행 또는 열 연산을 실시

① 원소추출

```
> mat <- matrix(c(1,2,3,4,5,6,7,8,9), ncol=3, byrow=T)
```

byrow=T 옵션을 줌으로써 행 기준 3열의 행렬 생성

```
> mat
```

```
[,1] [,2] [,3]
```

```
[1,] 1 2 3
```

```
[2,] 4 5 6
```

```
[3,] 7 8 9
```

3. 행렬

> mat[1,] # 행렬 mat의 1행의 값

[1] 1 2 3

> mat[,3] # 행렬 mat의 3열의 값

[1] 3 6 9

> mat[mat[,3] > 4, 2] # 3열에서 4보다 큰 행의 값 중 2열의 모든 값

[1] 5 8

> mat[2,3] # 2행 3열의 값 추출

[1] 6

3. 행렬

② apply() 함수

```
> Height <- c(140,155,142,175) # Height 벡터 생성
> size.1 <- matrix(c(130,26,110,24,118,25,112,25), ncol=2, byrow=T,
dimnames=list(c("Lee", "Kim", "Park", "Choi"), c("Weight", "Waist")))
# size.1 행렬을 생성하고 이름을 부여
> size <- cbind(size.1, Height) # size.1 행렬과 Height 벡터의 열 기준 결합
> size
```

	Weight	Waist	Height
Lee	130	26	140
Kim	110	24	155
Park	118	25	142
Choi	112	25	175

3. 행렬

```
> colmean <- apply(size, 2, mean)      # 2: 열의 평균값을 계산
```

```
> colmean
```

```
Weight Waist Height
```

```
117.5 25.0 153.0
```

```
> rowmean <- apply(size, 1, mean)  # 1: 행의 평균값(mean)을 계산
```

```
> rowmean
```

```
Lee Kim Park Choi
```

```
98.66667 96.33333 95.00000 104.00000
```

```
> colvar <- apply(size, 2, var)      # 2: 열의 분산값(var)을 계산
```

```
> colvar
```

```
Weight Waist Height
```

```
81.0000000 0.6666667 259.3333333
```

3. 행렬

```
> rowvar <- apply(size, 1, var)    # 1: 행의 분산값을 계산
```

```
> rowvar
```

```
Lee Kim Park Choi
```

```
3985.333 4430.333 3819.000 5673.000
```

3. 행렬

③ sweep 함수(기본연산은 “-”로 지정되어 있음)

> sweep(size, 2, colmean) # size 각 열의 값과 colmean의 차

Weight Waist Height

Lee 12.5 1 -13

Kim -7.5 -1 2

Park 0.5 0 -11

Choi -5.5 0 22

> sweep(size, 1, rowmean) # size 각 행의 값과 rowmean의 차

Weight Waist Height

Lee 31.33333 -72.66667 41.33333

Kim 13.66667 -72.33333 58.66667

Park 23.00000 -70.00000 47.00000

Choi 8.00000 -79.00000 71.00000

3. 행렬

```
> sweep(size, 1, c(1,2,3,4), "+") # size 각 행의 값에 c(1,2,3,4) 값을 더해 줌
```

```
Weight Waist Height
```

```
Lee 131 27 141
```

```
Kim 112 26 157
```

```
Park 121 28 145
```

```
Choi 116 29 179
```

```
> sweep(size, 1, c(1,2,3,4), "-") # size 각 행의 값에서 c(1,2,3,4) 값을 빼 줌
```

```
Weight Waist Height
```

```
Lee 129 25 139
```

```
Kim 108 22 153
```

```
Park 115 22 139
```

```
Choi 108 21 171
```

3. 행렬

<행렬의 연산과 관련된 주요 함수와 기능>

연산자 및 함수	기능	사용법
t	전치행렬을 구하는 함수	t(A)
% * %	행렬의 곱셈	A% * %B
crossprod	t(A)% * %B	crossprod(A,B)
outer	외적을 구하는 함수	outer(A,B)
svd	singular value 분해	svd(A)
qr	QR 분해함수	qr(A)
solve	방정식의 근을 구하는 함수	solve(A,B)
	역행렬을 구하는 함수	solve(A)
eigen	고유치와 고유벡터 함수	eigen(A)
chol	choleski 분해 함수	chol(A)

3. 행렬

예제 3-15 1에서 4까지의 값을 갖는 열 기준 행렬 m1과 5에서 8까지의 값을 갖는

열 기준의 행렬 m2를 생성하고 두 행렬의 곱, m1의 전치행렬 및 m1의 역행렬을 구하기

```
> m1 <- matrix(1:4, nrow=2)           # 1~4까지 2행 2열의 행렬 생성
```

```
> m1
```

```
[,1] [,2]
```

```
[1,] 1 3
```

```
[2,] 2 4
```

```
> m2 <- matrix(5:8, nrow=2)           # 5~8까지 2행 2열의 행렬 생성
```

```
> m2
```

```
[,1] [,2]
```

```
[1,] 5 7
```

```
[2,] 6 8
```

3. 행렬

```
> m1%*%m2
```

```
# m1과 m2 행렬의 곱셈
```

```
[,1] [,2]
```

```
[1,] 23 31
```

```
[2,] 34 46
```

```
> solve(m1)
```

```
# m1 행렬의 역 행렬 생성
```

```
[,1] [,2]
```

```
[1,] -2 1.5
```

```
[2,] 1 -0.5
```

```
> t(m1)
```

```
# m1 행렬의 전치행렬 생성
```

```
[,1] [,2]
```

```
[1,] 1 2
```

```
[2,] 3 4
```

04

실습하기

05강

다음시간안내 ▶▶▶

R레이터구조 (2)